use UltraChat [Ding et al., 2023] and XSTest [Röttger et al., 2023] for our benign dataset and the Circuit Breakers dataset [Zou et al., 2024] for harmful data. Unless stated otherwise, we use Llama-3-8B-Instruct as our model being monitored. When a probe requires a supervised loss to train we use a binary cross-entropy loss with harmfulness labels. For input-based probes we use the activations on the last input token, and for generation-based probes we use all activations on generation tokens. See Appendix A for details.

3.2 Ordinary Attacks

We investigate question (1), *can existing latent-space monitors reliably detect harmfulness against standard jailbreaking methods?* We provide evidence the answer is yes, demonstrating that standard jailbreaking techniques *do not* induce obfuscated activations against various monitors.

We create a dataset of successful jailbreaks by starting with our harmful dataset and using a variety of prominent jailbreaking methods to elicit harmful responses from the model.[1] We use GCG [Zou et al., 2023b], PAIR [Chao et al., 2023], MSJ [Anil et al., 2024], and AutoDAN [Liu et al., 2023], among others (see Appendix A.2 for further details). Note that probes are never trained on these jailbreak datasets, only on benign data and harmful data without jailbreaks. We evaluate how well probes transfer to distinguishing between the benign test split and harmful test inputs with jailbreaks.

Probes can reliably detect ordinary jailbreaks. As shown in Fig. 4a, the Logistic Regression and MLP probes can detect jailbreaks quite reliably, usually with 90% recall or more at a 1% false positive rate. Generation-based detection works best, and mean difference probes perform worse. See Table 1 in Appendix A.2 for a breakdown of probe performance by jailbreak type. This demonstrates that none of the jailbreaks we test strongly obfuscate activations.

3.3 Obfuscation Attacks

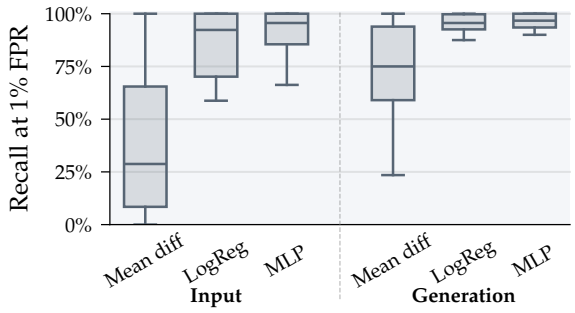
Given that existing jailbreak attacks do not bypass harmfulness probes, we turn to question (2), *can obfuscation attacks induce harmful activations that fool probes?*

Similar to the setting in §3.2, we seek to find adversarial suffixes to harmful requests that lead to a useful response from the model. The key difference is that we also optimize suffixes to fool a target probe, and thus seek to *explicitly induce obfuscated activations*.

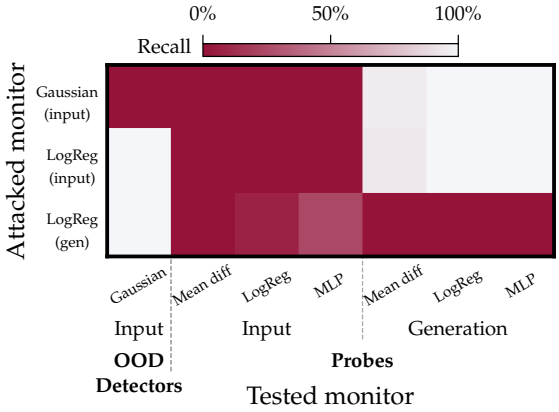
We study two different attacker affordances, hard prompt suffixes (real tokens appended to the end of an input) and embedding suffixes (a matrix of values appended to the end of a token sequence embedding [Lester et al., 2021]). While embedding suffix attacks are a strong affordance, they are a realistic threat model for an attacker with white-box access [Yang et al., 2021], and they provide insight into *worst-case* model performance under prompting [Zou et al., 2023b, Xhonneux et al., 2024, Che et al., 2024]. For hard prompt attacks, we optimize a new suffix for each input. For embedding attacks, we optimize a single *universal* suffix against a set of inputs, and test it on held-out inputs. We explored universal hard prompt attacks but did not find successful obfuscation attacks with them.

Training suffixes. To train an obfuscating and jailbreaking adversarial suffix w against some target probe D, we use an obfuscation attack (Eq. (1)) by jointly minimizing a behavior and an

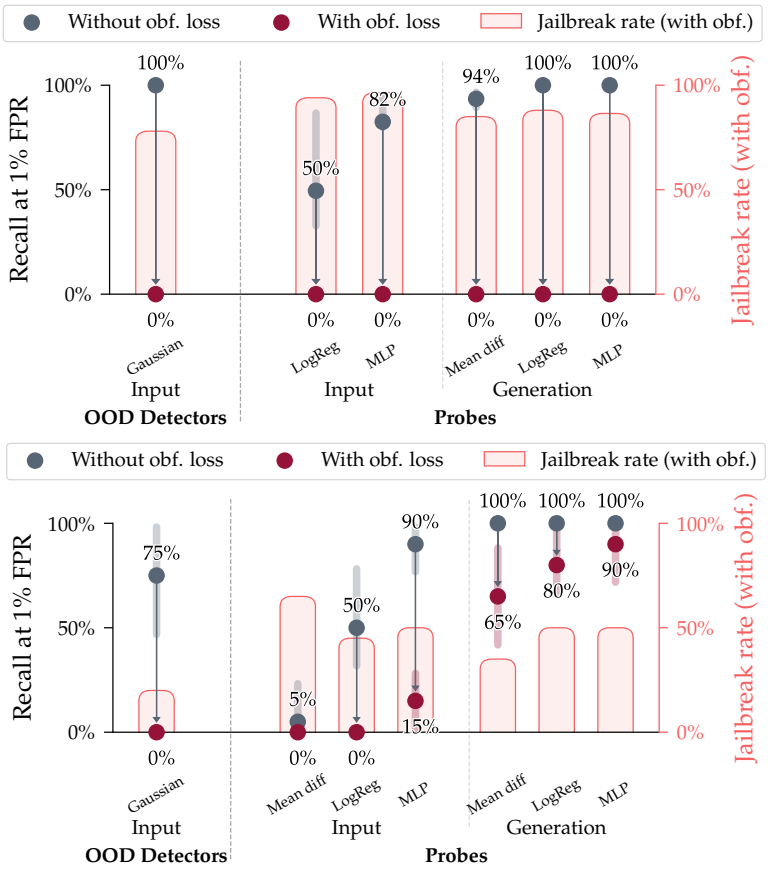
[1] We release all of these datasets at https://huggingface.co/datasets/Mechanistic-Anomaly-Detection/llama3-jailbreaks



(a) **Traditional jailbreaks *do not* reliably cause obfuscated activations.** Box plots show the performance distribution of probes against different types of *standard* jailbreaking techniques. Logistic regression and MLP probes achieve over 90% recall at a 1% false positive rate for the median jailbreak type.



(c) **Obfuscation attacks can transfer to other monitors**. This heat map shows transfer of obfuscation attacks trained against one probe and tested on another. Each row corresponds to one embedding suffix optimized against a monitor. Each column corresponds to a monitor we test this suffix against. Interestingly, optimizing against the Gaussian OOD detector also fools input-based probes, and optimizing against generation-based probes transfers to input-based ones. Full plot in Fig. 18.



(b) **Obfuscation attacks can bypass latent harmfulness monitors**. Here, we show the performance of obfuscation attacks at fooling the probe (dots and arrows) and eliciting the desired model behavior of jailbreaking (bar charts). Top, embedding suffix results. Bottom, input specific hard prompt results. **Baseline probe recall** against attack without explicit obfuscation loss is generally high. **Recall of probe under obfuscation** is significantly lower (error bars for recall calculated using bootstrapping). **Jailbreaking** rate under each obfuscated attack is shown as a bar plot. Hard token attacks aren't quite as successful but still achieve partial obfuscation for all probes.

Figure 4: **Standard jailbreaking and obfuscation attack results against Llama-3-8b-Instruct.**

obfuscation loss:

$$\mathcal{L}(w) = \frac{1}{|\mathcal{D}|} \sum_{\texttt{inp},\texttt{out} \in \mathcal{D}} [\underbrace{\text{CrossEntropy}(M(\texttt{inp} \oplus w), \texttt{out})}_{\text{Behavior Loss}} + \underbrace{D(\mathcal{R}(\texttt{inp} \oplus w \oplus \texttt{out}))}_{\text{Obfuscation Loss}}], \quad (2)$$

where $\mathcal{D}$ is a set of harmful requests `inp` and responses `out`, and $\oplus$ means concatenation of tokens or embeddings along the sequence dimension. If w is to be a universal suffix (that is, we intend w to be used for *any* input), we use our harmful dataset for $\mathcal{D}$. In contrast, if w is input-specific, then it is designed to work against a specific harmful request and $\mathcal{D}$ contains only that one request.

For embedding suffixes, we assume the attacker has white-box access to the model and use the Adam optimizer to minimize Eq. (2) with a fixed suffix token length of 16. For hard prompts, we only give the attacker access to the target model logit outputs and probe output (commonly referred to as gray-box access). Thus, our hard prompt suffixes are created in a *gradient-free* manner. We use a modified version of the FLRT optimizer [Thompson and Sklar, 2024]. In particular, we replace the worst candidate string from the attack buffer as opposed to the best, and also remove the perplexity penalty, since we focus on attacking latent monitors. We optimize an adversarial suffix against 20 harmful queries separately. This small number induces finite sample error in the results but we report 95% bootstrapped confidence intervals to account for this. The FLRT optimizer can add and remove tokens to the suffix during training, so we do not place restrictions on the hard prompt suffix length.

Probes perform well on clean prompts but get 0% recall against obfuscation attacks. All probes perform well at the base task of distinguishing between harmful and benign activations. Despite the strong performance on standard inputs, obfuscation embedding attacks and even hard prompts induce obfuscated activations against a wide range of monitors while still jailbreaking the model (Fig. 4b).

Obfuscation attacks often transfer between defenses. In Fig. 4c, we show the results of training a suffix against one probe and then evaluating it against a different type of probe (for figure showing all combinations of train and test probe see Fig. 18). We observe a remarkable amount of transfer; for example, training against a logistic regression probe also bypasses MLP probes. Training against generation-based probes bypasses input-based probes but not vice versa. There is no transfer from training against probes to testing against OOD detectors. But perhaps most strikingly, we do observe transfer in the reverse direction: training against an OOD detector bypasses input-based probes, even though these are a very different approach to harmfulness monitoring. This supports the intuition we presented in Fig. 2.

These transfer results suggest shared weaknesses between different latent monitoring methods. This also means that an attacker does not necessarily need to know which monitoring methods a defender employs; obfuscating against one of them might also bypass many others.

3.4 Vulnerabilities in Circuit Breakers and LAT

Two recent methods, Circuit Breakers [Zou et al., 2024] and Latent Adversarial Training (LAT) [Sheshadri et al., 2024, Casper et al., 2024], attempt to increase LLM robustness by applying losses directly to activations during training. Circuit Breakers scramble activations on harmful inputs to prevent coherent generations, whereas LAT trains the model to be robust to perturbations in